

Практика 1-2 Minimal API(Task Manager)

Практическая работа

Разработка REST API на ASP.NET Core (Minimal API)

Цель работы

Изучить основы разработки backend-приложений на платформе **.NET** с использованием **ASP.NET Core Minimal API**.

В ходе работы необходимо разработать **REST API для управления списком задач**.

 В данной работе **база данных не используется**.
Все данные должны храниться в **JSON файле на сервере**.

Описание задания

Необходимо реализовать **REST API "Менеджер задач" (Task Manager API)**.

Система должна позволять:

- получать список задач
- получать конкретную задачу
- создавать новую задачу
- изменять задачу
- удалять задачу

Все данные должны сохраняться в **JSON-файле**.

Хранение данных

Создайте файл:

Data/tasks.json

Начальное содержимое файла:

```
[ ]
```

Этот файл будет использоваться как **простое хранилище данных (имитация базы данных)**.

Все задачи должны:

- загружаться из этого файла
- сохраняться обратно в файл после изменений

Архитектурное правило

Endpoint (например `/api/tasks`) **не является местом хранения данных**.

Endpoint - это обработчик HTTP-запроса.

Архитектура backend:

Клиент → HTTP запрос → Endpoint → чтение JSON → ответ клиенту

Требуемые API endpoints

1. Получить список всех задач

GET `/api/tasks`

Пример ответа:

```
[
  {
    "id": 1,
    "title": "Сделать практичку",
    "description": "ASP.NET Core API",
    "isCompleted": false
  }
]
```

2. Получить задачу по ID

GET /api/tasks/{id}

Если задача не найдена --- вернуть:

404 Not Found

3. Создать новую задачу

POST /api/tasks

Тело запроса:

```
{
  "title": "Изучить ASP.NET Core",
  "description": "Minimal API"
}
```

Сервер должен:

- создать новую задачу
 - автоматически сгенерировать Id
 - сохранить задачу в JSON файл
-

4. Обновить задачу

PUT /api/tasks/{id}

```
{  
  "title": "Изучить ASP.NET Core",  
  "description": "Minimal API и Swagger",  
  "isCompleted": true  
}
```

5. Удалить задачу

DELETE /api/tasks/{id}

После удаления задача должна исчезнуть из JSON файла.

Работа с JSON

Используйте библиотеку:

System.Text.Json

Пример чтения JSON

```
var json = File.ReadAllText("Data/tasks.json");  
var tasks = JsonSerializer.Deserialize<List<TaskModel>>(json);
```

Пример записи JSON

```
var json = JsonSerializer.Serialize(tasks);  
File.WriteAllText("Data/tasks.json", json);
```

Требования к реализации

В проекте должны быть реализованы:

- ✓ модель данных
 - ✓ JSON файл для хранения данных
 - ✓ endpoints для CRUD операций
 - ✓ использование Minimal API
 - ✓ тестирование API через Swagger
-

Проверка работы API

Проверить работу API можно через:

- Swagger UI
 - Postman
 - браузер (для GET запросов)
-

Пример сценария работы

1. Создать задачу
POST /api/tasks
 2. Получить список задач
GET /api/tasks
 3. Изменить задачу
PUT /api/tasks/1
 4. Удалить задачу
DELETE /api/tasks/1
-

Дополнительное задание (по желанию)

Реализовать endpoint:

GET /api/tasks/completed

Он должен возвращать **только выполненные задачи**.

Критерии оценки

Критерий	Баллы
Модель данных	1
Работа с JSON	2
GET endpoints	2
POST endpoint	2
PUT endpoint	2
DELETE endpoint	1
Доп.задание	5

Максимум: **15 баллов**

Результат

В результате должен получиться **REST API сервер**, который:

- принимает HTTP запросы
- работает с JSON
- реализует CRUD операции
- хранит данные в файле